#D Add the token to the URI as a query parameter.

You can now wire up code to create the `CapabilityController` inside your main method, so open Main.java in your editor and create a new instance of the object along with a token store for it to use. You can use any secure token store implementation, but for this chapter you'll use the `DatabaseTokenStore` because it creates short tokens and therefore short URIs. You should also pass the new controller as an additional argument to the `SpaceController` constructor, because you will shortly use it to create capability URIs:

```
var database = Database.forDataSource(datasource);
var capController = new CapabilityController(
new DatabaseTokenStore(database));
var spaceController = new SpaceController(database,
capController);
var userController = new UserController(database);
```

Before you can start generating capability URIs though, you need to make one tweak to the database token store. The current store requires that every token has an associated user and will raise an error if you try to save a token with a `null` username. Because capabilities are not identity-based, you need to remove this restriction. Open `schema.sql` in your editor and remove the not-null constraint from the `tokens` table by deleting the words `NOT NULL` from the end of the `user_id` column definition. The new table definition should look like the following:

```
CREATE TABLE tokens(
token_id VARCHAR(30) PRIMARY KEY,
```